

PyGNSS-RT

Technical Design Document

Python-Based Real-Time PPP-AR GNSS Positioning System

Technical Specification and Architecture

Repository: https://github.com/georgisalene/pygnss_rt

Version: v1.4.0-GRE

Configuration: GPS + GLONASS + Galileo (GRE)

Addisu Hunegnaw

GNSS Research Group

Institute for Earth Sciences and Space Geodesy

`pygnss_rt` — Python GNSS Real-Time Processing

January 2026

Abstract

This document provides a comprehensive technical specification for the `pygnss_rt` Python package, a production-grade framework for Real-Time Precise Point Positioning with Ambiguity Resolution (PPP-AR). The system integrates with the Bernese GNSS Software v5.4 for core geodetic computations while providing a sophisticated Python orchestration layer for data management, product acquisition, and operational automation.

Key Features:

- Multi-GNSS support: GPS (G) + GLONASS (R) + Galileo (E) triple-constellation
- VMF3 Vienna Mapping Functions for tropospheric modeling
- CODE OSB (Observable-Specific Signal Biases) for phase bias consistency
- LAMBDA-based integer ambiguity resolution
- Automated product downloads from CDDIS, CODE, IGN
- ZTD to IWV conversion for meteorological applications
- Five international station networks (IGS, EUREF, GB, RGP, Supersites)

GitHub Repository: https://github.com/georgisalene/pygnss_rt

This document describes the system architecture, mathematical formulations, processing workflows, and configuration options.

Contents

1	Introduction	4
1.1	Document Purpose and Scope	4
1.1.1	What This Document Provides	4
1.2	System Overview	4
1.3	GRE Configuration	4
2	System Architecture	6
2.1	Package Structure	6
2.2	Data Flow Pipeline	7
3	Command-Line Interface	8
3.1	Overview	8
3.2	Available Commands	8
3.3	Command Options	8
3.3.1	daily-ppp Command	8
4	Daily PPP Processor	9
4.1	Module Overview	9
4.2	Processing Workflow	9
4.3	Result Structure	9
5	Product Downloader	10
5.1	Module Overview	10
5.2	Supported Products	10
5.3	Multi-Source Retry Logic	10
6	Bernese Processing Engine Runner	11
6.1	Module Overview	11
6.2	BPE Execution Workflow	11
6.3	INP File Customization	11
6.4	BPE Configuration Parameters	11
7	Network Profiles	12
7.1	Module Overview	12
7.2	Supported Networks	12
7.3	Network Profile Components	12
8	Date and Time Utilities	13
8.1	GNSSDate Class	13
8.2	Time System Constants	13
8.3	Conversion Methods	13
9	ZTD to IWV Conversion	14

9.1	Physical Background	14
9.1.1	Mathematical Formulation	14
9.2	Physical Constants	14
10	Station Management	15
10.1	Station Class	15
10.2	Station Attributes	15
10.3	Station File Formats	15
11	Mathematical Foundations	16
11.1	PPP-AR Observation Model	16
11.2	Kalman Filter State Vector	16
11.3	LAMBDA Ambiguity Resolution	16
11.4	VMF3 Mapping Functions	17
12	Bernese Configuration	18
12.1	YAML Configuration Format	18
12.2	GRE Configuration Rationale	19
A	Module Summary	20
B	Installation and Usage	21
B.1	Installation from GitHub	21
B.2	Basic Usage Examples	21
C	References	22

List of Figures

2.1	PyGNSS-RT Package Architecture	6
2.2	PPP-AR Processing Data Flow	7

List of Tables

1.1	PyGNSS-RT System Capabilities	4
1.2	GRE Processing Configuration	5
3.1	CLI Commands Overview	8
3.2	daily-ppp Command Options	8
4.1	DailyPPPResult Fields	9
5.1	GNSS Product Types	10

6.1	BPEConfig Parameters	11
7.1	Network Identifiers	12
8.1	GNSS Time Constants	13
8.2	GNSSDate Conversion Methods	13
9.1	Physical Constants (Bevis, 1994)	14
10.1	Station Class Attributes	15
12.1	GRE (GPS+GLONASS+Galileo) Configuration	19
A.1	PyGNSS-RT Main Modules	20

Chapter 1

Introduction

1.1 Document Purpose and Scope

This technical design document serves as the authoritative reference for the `pygnss_rt` real-time GNSS processing system. The complete source code is available at:

GitHub Repository

https://github.com/georgisalene/pygnss_rt

1.1.1 What This Document Provides

- **Complete mathematical formulations** for PPP-AR processing
- **System architecture** and data flow specifications
- **Implementation details** for atmospheric and bias modeling
- **Ambiguity resolution algorithms** and validation strategies
- **Configuration reference** for Bernese integration

1.2 System Overview

The `pygnss_rt` package implements a sophisticated orchestration framework for GNSS data processing:

Table 1.1: PyGNSS-RT System Capabilities

Feature	Implementation	Status
Multi-GNSS Processing	GPS + GLONASS + Galileo (GRE)	✓ Active
PPP-AR	CODE OSB + LAMBDA	✓ Active
Troposphere Modeling	VMF3 + ZTD estimation	✓ Active
Product Downloads	CDDIS, CODE, IGN	✓ Active
IWV Derivation	Bevis formulation	✓ Active
Network Support	5 networks (IG/EU/GB/RG/SS)	✓ Active
GLONASS AR	GLORES: ALWAYS	✓ Active
BeiDou AR	DCB complexity	× Not implemented

1.3 GRE Configuration

The system uses a triple-constellation configuration with special handling for the BSW 5.4 CODSPP 2-GNSS limitation:

Table 1.2: GRE Processing Configuration

Program	GPS	GLONASS	Galileo
CODSPP (clock sync)	✓	–	–
GPSEST (PPP-AR)	✓	✓	✓

This approach mirrors the proven double-difference processing strategy where CODSPP uses GPS-only for initial clock synchronization, while GPSEST performs full GRE processing.

Chapter 2

System Architecture

2.1 Package Structure

The `pygnss_rt` package is organized into functional modules:

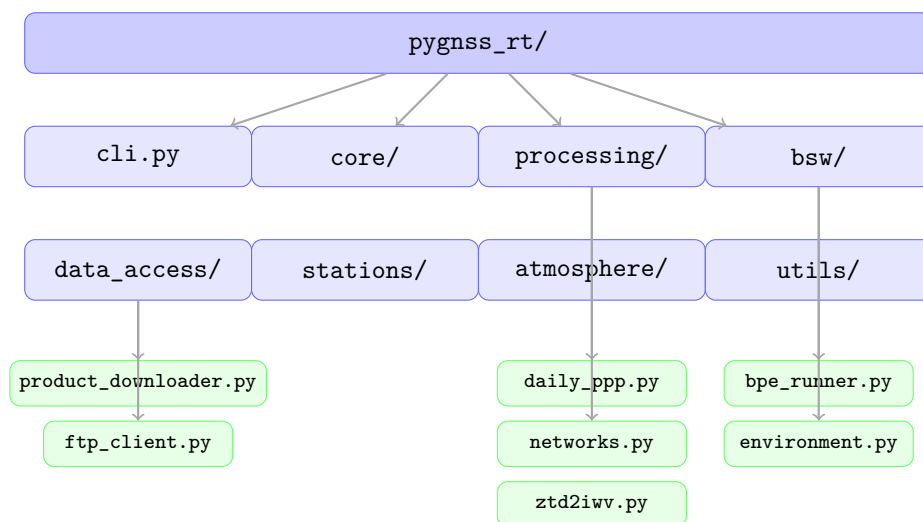


Figure 2.1: PyGNSS-RT Package Architecture

2.2 Data Flow Pipeline

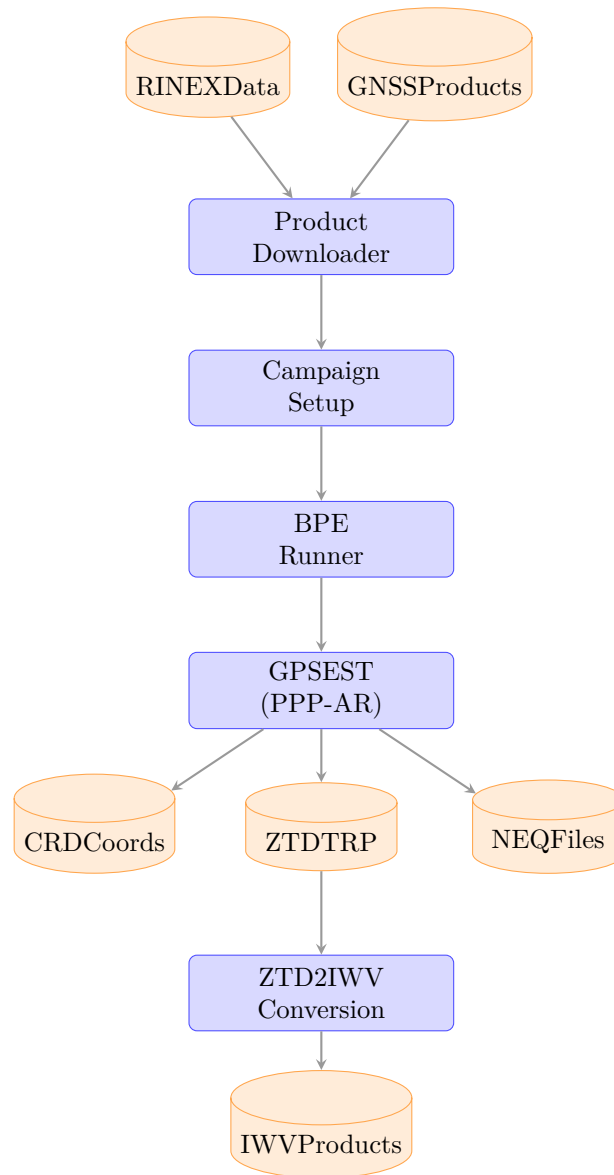


Figure 2.2: PPP-AR Processing Data Flow

Chapter 3

Command-Line Interface

3.1 Overview

The CLI module (`cli.py`) provides a modern Click-based interface for all PyGNSS-RT operations.

3.2 Available Commands

Table 3.1: CLI Commands Overview

Command	Usage	Description
<code>daily-ppp</code>	<code>daily-ppp IG -s DATE -e DATE</code>	Run daily PPP-AR processing
<code>nrddp-tro</code>	<code>nrddp-tro -s DATE -e DATE</code>	NRDDP tropospheric processing
<code>download-products</code>	<code>download-products -d DATE</code>	Download GNSS products
<code>ztd2iwv</code>	<code>ztd2iwv FILE</code>	Convert ZTD to IWV
<code>stations</code>	<code>stations FILE</code>	List stations from config
<code>update-sta</code>	<code>update-sta -n NETWORK</code>	Update BSW STA file

3.3 Command Options

3.3.1 daily-ppp Command

Table 3.2: daily-ppp Command Options

Option	Type	Description
<code>NETWORK</code>	Choice	Network identifier (IG, EU, GB, RG, SS)
<code>-s, -start-date</code>	String	Start date (YYYY-MM-DD)
<code>-e, -end-date</code>	String	End date (YYYY-MM-DD)
<code>-S, -stations</code>	String	Comma-separated station list
<code>-x, -exclude</code>	String	Stations to exclude
<code>-skip-products</code>	Flag	Skip product download
<code>-skip-data</code>	Flag	Skip station data download
<code>-skip-dcm</code>	Flag	Skip DCM archiving
<code>-dry-run</code>	Flag	Show what would be done

Chapter 4

Daily PPP Processor

4.1 Module Overview

The daily PPP processor ([processing/daily_ppp.py](#)) implements the unified daily PPP-AR processing workflow, replacing five separate Perl caller scripts:

- `iGNSS_D_PPP_AR_IG_IGS54_direct_NRT.pl` – IGS stations
- `iGNSS_D_PPP_AR_EU_IGS54_direct_NRT.pl` – EUREF stations
- `iGNSS_D_PPP_AR_GB_IGS54_direct_NRT.pl` – GB stations
- `iGNSS_D_PPP_AR_RG_IGS54_direct_NRT.pl` – RGP France stations
- `iGNSS_D_PPP_AR_SS_IGS54_direct_NRT.pl` – Supersites

4.2 Processing Workflow

The daily PPP processor follows this workflow:

1. **Download Products** – Orbit (SP3), Clock (CLK), ERP, BIA/OSB, VMF3
2. **Download Station Data** – RINEX observations from FTP servers
3. **Setup Campaign** – Create BSW campaign directory structure
4. **Run BSW Processing** – Execute BPE with GPSEST for PPP-AR
5. **DCM Archiving** – Archive results and clean up

4.3 Result Structure

Table 4.1: DailyPPPResult Fields

Field	Type	Description
<code>network_id</code>	<code>str</code>	Network identifier
<code>session_name</code>	<code>str</code>	Session name (e.g., 25357IG)
<code>date</code>	<code>GNSSDate</code>	Processing date
<code>success</code>	<code>bool</code>	Processing success flag
<code>stations_processed</code>	<code>int</code>	Number of stations processed
<code>duration_seconds</code>	<code>float</code>	Processing duration
<code>output_files</code>	<code>list</code>	List of output file paths

Chapter 5

Product Downloader

5.1 Module Overview

The product downloader ([data_access/product_downloader.py](#)) replaces the Perl FTP.pm module, providing unified download capabilities for all GNSS products.

5.2 Supported Products

Table 5.1: GNSS Product Types

Product	Format	Sources	Purpose
Orbit	SP3	CDDIS, IGN, CODE	Satellite positions
Clock	CLK	CDDIS, IGN, CODE	Satellite clock corrections
ERP	ERP	CDDIS, IGN, CODE	Earth rotation parameters
BIA/OSB	BIA	CDDIS, CODE	Signal biases for PPP-AR
VMF3	GRD	TU Wien	Troposphere mapping functions
ION	ION	CDDIS, CODE	Ionosphere maps

5.3 Multi-Source Retry Logic

The downloader implements automatic retry logic across multiple servers:

1. Try primary server (e.g., CDDIS)
2. On failure, try secondary server (e.g., IGN)
3. On failure, try tertiary server (e.g., CODE)
4. Handle compression (.Z, .gz, .zip) automatically

Chapter 6

Bernese Processing Engine Runner

6.1 Module Overview

The BPE runner ([bsw/bpe_runner.py](#)) manages Bernese GNSS Software execution, replacing the Perl startBPE.pm and RUNBPE.pm modules.

6.2 BPE Execution Workflow

1. Create temporary user area ($\$T/BPE_pcf_year_session_pid$)
2. Copy INP files from OPT directories to temp $\$U/PAN/$
3. Customize INP files with putKey operations
4. Create MENU_\$\$TMP control file
5. Execute menu.sh to run BPE scripts
6. Monitor output and wait for completion
7. Clean up temporary files

6.3 INP File Customization

The put_key function mimics the Perl putKey functionality:

- Reads INP file content
- Matches key pattern: KEY "value" or KEY 'value'
- Replaces value while preserving quotes
- Writes updated content back to file

6.4 BPE Configuration Parameters

Table 6.1: BPEConfig Parameters

Parameter	Type	Description
pcf_file	str	PCF file name (without path)
campaign	str	Campaign name/path
session	str	Session identifier
year	int	Processing year
task_id	str	2-char task identifier
cpu_file	str	CPU configuration file
max_time	int	Timeout in seconds (0=no limit)

Chapter 7

Network Profiles

7.1 Module Overview

The networks module ([processing/networks.py](#)) defines configuration profiles for five international GNSS station networks.

7.2 Supported Networks

Table 7.1: Network Identifiers

ID	Name	Description
IG	IGS	IGS core stations (global reference network)
EU	EUREF	EUREF stations (European reference network)
GB	Great Britain	OS active, scientific, local IGS stations
RG	RGP	RGP France stations (French permanent network)
SS	Supersites	Netherlands/European supersites

7.3 Network Profile Components

Each network profile contains:

- **Station Filter** – XML/YAML file, network filter, station type
- **Data FTP Sources** – Server IDs and data types
- **Product Sources** – Orbit, ERP, clock provider configuration
- **BSW Configuration** – PCF file, YAML options file
- **Archive Settings** – DCM directories, file specifications

Chapter 8

Date and Time Utilities

8.1 GNSSDate Class

The dates module ([utils/dates.py](#)) provides conversions between various GNSS time systems:

- Modified Julian Date (MJD)
- GPS Week and Day of Week
- Year and Day of Year (DOY)
- Calendar dates (YYYY-MM-DD)
- Hour alpha characters (a-x for 0-23)

8.2 Time System Constants

Table 8.1: GNSS Time Constants

Constant	Value	Description
GPS_EPOCH_MJD	44244.0	January 6, 1980
MJD_OFFSET	2400000.5	Offset from JD to MJD

8.3 Conversion Methods

Table 8.2: GNSSDate Conversion Methods

Method	Description
<code>from_mjd(mjd)</code>	Create from Modified Julian Date
<code>from_gps_week(week, dow)</code>	Create from GPS week and day of week
<code>from_doy(year, doy)</code>	Create from year and day of year
<code>from_string(date_str)</code>	Parse from YYYY-MM-DD or YYYY/DOY
<code>add_days(n)</code>	Add n days to date
<code>add_hours(n)</code>	Add n hours to date

Chapter 9

ZTD to IWV Conversion

9.1 Physical Background

The ZTD to IWV conversion implements the Bevis (1992, 1994) formulation for deriving Integrated Water Vapor from GNSS-estimated Zenith Tropospheric Delay.

9.1.1 Mathematical Formulation

The Zenith Total Delay (ZTD) is decomposed into hydrostatic and wet components:

$$\text{ZTD} = \text{ZHD} + \text{ZWD} \quad (9.1)$$

The Zenith Hydrostatic Delay is computed using Saastamoinen (1972):

$$\text{ZHD} = \frac{0.0022768 \cdot P_s}{1 - 0.00266 \cos(2\phi) - 0.00028H} \quad (9.2)$$

The IWV is then derived from ZWD using:

$$\text{IWV} = \frac{10^6 \cdot \text{ZWD}}{\rho_w \cdot R_v \cdot (k'_2 + k_3/T_m)} \quad (9.3)$$

where T_m is the mean atmospheric temperature estimated from surface temperature using the Bevis empirical formula:

$$T_m = 70.2 + 0.72 \cdot T_s \quad (9.4)$$

9.2 Physical Constants

Table 9.1: Physical Constants (Bevis, 1994)

Symbol	Value	Unit	Description
R_{dry}	287.0586	J/kg/K	Gas constant for dry air
R_{vapor}	461.525	J/kg/K	Gas constant for water vapor
k_1	77.6	K/hPa	Refractivity constant
k_2	70.4	K/hPa	Refractivity constant
k_3	373900	K ² /hPa	Refractivity constant
g	9.80665	m/s ²	Standard gravity
α	-0.0065	K/m	Temperature lapse rate

Chapter 10

Station Management

10.1 Station Class

The station module ([stations/station.py](#)) provides unified station management supporting both XML and YAML formats.

10.2 Station Attributes

Table 10.1: Station Class Attributes

Attribute	Type	Description
station_id	str	4-character station ID
name	str	Station name
network	str	Network identifier
latitude	float	Latitude (degrees)
longitude	float	Longitude (degrees)
height	float	Ellipsoidal height (m)
x, y, z	float	ECEF coordinates (m)
receiver_type	str	Receiver type
antenna_type	str	Antenna type
use_nrt	bool	NRT processing enabled
active	bool	Station active flag

10.3 Station File Formats

The system supports two station file formats:

- **YAML** (preferred) – Human-readable, easy to edit
- **XML** (legacy) – Compatible with existing infrastructure

Chapter 11

Mathematical Foundations

11.1 PPP-AR Observation Model

The ionosphere-free linear combination for dual-frequency observations:

$$L_{IF} = \frac{f_1^2 L_1 - f_2^2 L_2}{f_1^2 - f_2^2} \quad (11.1)$$

$$P_{IF} = \frac{f_1^2 P_1 - f_2^2 P_2}{f_1^2 - f_2^2} \quad (11.2)$$

11.2 Kalman Filter State Vector

The PPP-AR state vector contains:

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \delta t_r \\ \text{ZTD} \\ G_N, G_E \\ \mathbf{N} \end{bmatrix} \quad (11.3)$$

where:

- $\mathbf{r}$: Position coordinates (X, Y, Z)
- δt_r : Receiver clock offset
- ZTD: Zenith Tropospheric Delay
- G_N, G_E : Tropospheric gradients (North, East)
- $\mathbf{N}$: Carrier phase ambiguities

11.3 LAMBDA Ambiguity Resolution

The integer ambiguity search minimizes:

$$\|\hat{\mathbf{a}} - \mathbf{a}\|_{\mathbf{Q}_{\hat{\mathbf{a}}}}^2 = (\hat{\mathbf{a}} - \mathbf{a})^T \mathbf{Q}_{\hat{\mathbf{a}}}^{-1} (\hat{\mathbf{a}} - \mathbf{a}) \quad (11.4)$$

The ratio test validates the solution:

$$R = \frac{\|\hat{\mathbf{a}} - \mathbf{a}_2\|^2}{\|\hat{\mathbf{a}} - \mathbf{a}_1\|^2} > R_{threshold} \quad (11.5)$$

11.4 VMF3 Mapping Functions

The Vienna Mapping Functions VMF3 compute slant delays:

$$m_h(e) = \frac{1 + \frac{a_h}{1 + \frac{b_h}{c_h}}}{\sin e + \frac{a_h}{\sin e + \frac{b_h}{c_h}}} \quad (11.6)$$

$$m_w(e) = \frac{1 + \frac{a_w}{1 + \frac{b_w}{c_w}}}{\sin e + \frac{a_w}{\sin e + \frac{b_w}{c_w}}} \quad (11.7)$$

Chapter 12

Bernese Configuration

12.1 YAML Configuration Format

The BSW configuration uses YAML format for INP file customization:

Listing 12.1: PPP-AR YAML Configuration Example

```
1  # PyGNSS-RT BSW Configuration for PPP-AR (GRE Mode)
2  # Network: IGS Core Stations
3  # Version: 1.4.0-GRE
4
5  processing_mode: PPP_AR
6  gnss_systems:
7    GPS: true
8    Galileo: true
9    GLONASS: true # Enabled in GPSEST, disabled in CODSP
10
11 bern_options:
12   PPP_GEN:
13     CODSP:
14       # GPS-only for clock synchronization (BSW 5.4 limit)
15       USE_G: '1'
16       USE_R: '0'
17       USE_E: '0'
18
19     GPSEST:
20       # Full GRE for PPP-AR processing
21       USE_G: '1'
22       USE_R: '1'
23       USE_E: '1'
24       USE_C: '0'
25
26       # Troposphere modeling
27       TROPOS: DRY_VMF3
28       TROPOS_S: MENDES-PAVLIS
29
30       # Ambiguity resolution
31       AMBSTRAT: SIGMA
32       USEAR_G: '1'
33       USEAR_E: '1'
34       USEAR_R: '1'
35       GLORES: ALWAYS
36       LAMRES: GNSS_BY_GNSS
37
38       # OSB configuration
39       OSBREC_G: SYSTEM
40       OSBREC_E: SYSTEM
41       OSBREC_R: SATELLITE
42
43 PPP_IAR:
```

```

44 GPSEST:
45   # Integer Ambiguity Resolution step
46   AMBSTRAT: SIGMA
47   USEAR_G: '1'
48   USEAR_E: '1'
49   USEAR_R: '1'
50
51   # Ratio test threshold
52   MAXFRAC: '0.1'
53   SEARWID: '0.5'
54   RMSNARR: '0.03'

```

12.2 GRE Configuration Rationale

Table 12.1: GRE (GPS+GLONASS+Galileo) Configuration

Aspect	Setting	Rationale
CODSPP GNSS	GPS-only	BSW 5.4 CODSPP has 2-GNSS limit; GPS sufficient for clock sync
GPSEST GNSS	GRE (all 3)	Full triple-constellation for maximum redundancy
GLORES	ALWAYS	Enable GLONASS AR across different FDMA frequencies
AR Method	GNSS_BY_GNSS	Independent AR per constellation then combine
OSBREC_R	SATELLITE	Per-satellite OSB for GLONASS FDMA IFBs
Troposphere	VMF3	Vienna Mapping Functions for accurate ZTD

Appendix A

Module Summary

Table A.1: PyGNSS-RT Main Modules

Package	Module	Purpose
pygnss_rt/	cli.py	Command-line interface
core/	paths.py config.py exceptions.py	Path configuration System configuration Custom exceptions
processing/	daily_ppp.py networks.py nrddp_tro.py	Daily PPP processor Network profiles NRDDP troposphere
data_access/	product_downloader.py station_downloader.py ftp_client.py	Product downloads Station data downloads FTP client
bsw/	bpe_runner.py environment.py	BPE execution BSW environment
atmosphere/	ztd2iwv.py	ZTD to IWV conversion
stations/	station.py	Station management
utils/	dates.py logging.py	Date utilities Logging configuration

Appendix B

Installation and Usage

B.1 Installation from GitHub

Listing B.1: Installation Commands

```
1 # Clone repository
2 git clone https://github.com/georgisalene/pygnss_rt.git
3 cd pygnss_rt
4
5 # Create virtual environment
6 python3 -m venv .venv
7 source .venv/bin/activate
8
9 # Install package
10 pip install -e .
11
12 # Verify installation
13 pygnss_rt --version
```

B.2 Basic Usage Examples

Listing B.2: CLI Usage Examples

```
1 # Process IGS network for a date range
2 pygnss_rt daily-ppp IG -s 2025-12-01 -e 2025-12-31
3
4 # Download products only
5 pygnss_rt download-products -d 2025-12-23
6
7 # Convert ZTD to IWV
8 pygnss_rt ztd2iwv /path/to/troposphere.TRP
9
10 # List stations in a network
11 pygnss_rt stations /path/to/IGS20.yaml --network IGS20
12
13 # Update BSW station file
14 pygnss_rt update-sta -n IG --output NEWNRT54.STA
```

Appendix C

References

1. Bevis, M., Businger, S., Herring, T.A., Rocken, C., Anthes, R.A., Ware, R.H. (1992). GPS Meteorology: Remote Sensing of Atmospheric Water Vapor Using the Global Positioning System. *Journal of Geophysical Research*, 97(D14), 15787-15801.
2. Landskron, D., Böhm, J. (2018). VMF3/GPT3: Refined discrete and empirical troposphere mapping functions. *Journal of Geodesy*, 92, 349-360.
3. Teunissen, P.J.G. (1995). The least-squares ambiguity decorrelation adjustment: a method for fast GPS integer ambiguity estimation. *Journal of Geodesy*, 70, 65-82.
4. Dach, R., Lutz, S., Walser, P., Fridez, P. (2015). Bernese GNSS Software Version 5.2. *Astronomical Institute, University of Bern*.
5. Schaer, S. (2016). CODE's observable-specific signal bias (OSB) files: PPP-AR ready. *IGS Workshop 2016, Sydney*.